



cocotb

Проектирование цифровых вычислительных систем
Прутьянов В. В.



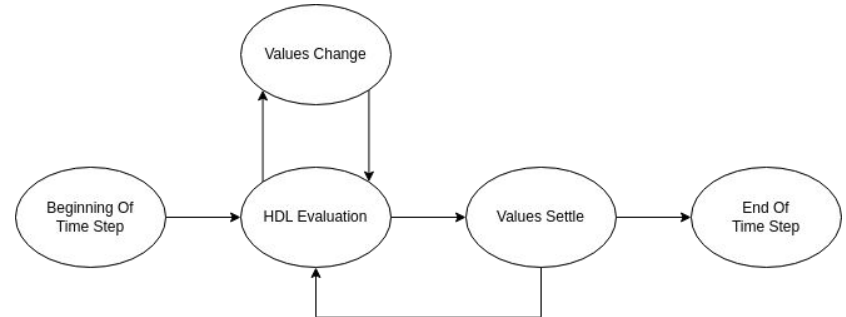
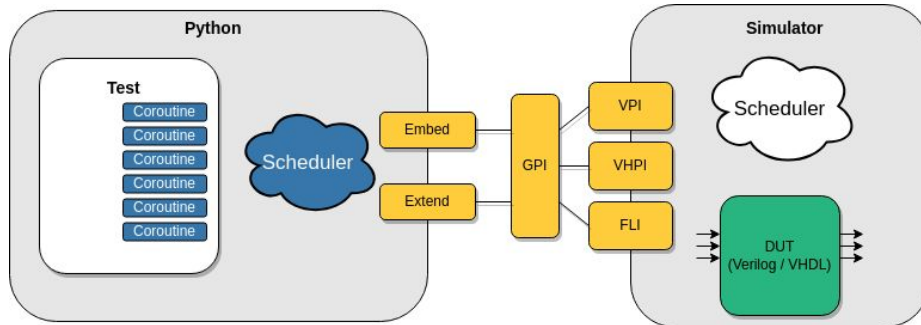
cocotb

- Фреймворк для верификации RTL-код при помощи Python
- cocotb – COroutine-based COsimulation TestBench
- Verilog/SystemVerilog/VHDL используются только для реализации тестируемого блока
- Python используется для реализации тестового окружения
- Фреймворк поддерживает различные симуляторы, в т.ч. open-source
- В тестбенче можно использовать любые возможности языка Python



cocotb

- Каждый тест это функция (coroutine) на языке Python
- Device Under Test (DUT), написанный на HDL, инстанцируется на верхнем уровне иерархии модулей
- Код на Python (через cocotb) управляет входными сигналами DUT и следит за выходными сигналами
- В каждый момент реального времени выполняется либо HDL-код в RTL-симуляторе (в симуляторе идет время), либо Python-код (симуляция стоит на паузе)
- Ключевое слово `await` используется внутри корутины для блокировки и ожидания другой корутины или возврата управления в RTL-симулятор для продолжения симуляции
- В тесте могут работать несколько корутин одновременно*, например, обрабатывая разные сигналы



DUT: counter.sv

```
module counter #(
    parameter WIDTH = 6
) (
    input  logic clk,
    input  logic rst_n,

    output logic [WIDTH-1:0] o_cnt
);

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        o_cnt <= '0;
    else
        o_cnt <= o_cnt + 1'b1;
end

endmodule
```

TB: counter_tb.py

```
import cocotb
from cocotb.triggers import RisingEdge
from cocotb.clock import Clock

@cocotb.test() # Добавляет тест в список тестов для запуска
async def simple(dut):

    # Создаем объект корутины, переключающей сигнал clk с частотой 1ГГц
    clock = Clock(dut.clk, 1, "ns").start(start_high=False)
    # Планируем запуск корутины, отложенный до следующего await
    cocotb.start_soon(clock)

    dut.rst_n.value = 0      # Записываем 0 в rst_n
    await RisingEdge(dut.clk) # Передаем управление симулятору до след. фронта clk
    dut.rst_n.value = 1      # Записываем 1 в rst_n
    await RisingEdge(dut.clk) # Передаем управление симулятору до след. фронта clk

    for i in range(10):
        cocotb.log.info(f"Counter: {dut.o_cnt.value}") # Считываем значение o_cnt
        await RisingEdge(dut.clk) # Передаем управление симулятору до след. фронта clk

    assert dut.o_cnt.value == 10, f"Value is {dut.o_cnt.value}" # Проверка
```

Makefile

```
SIM ?= verilator # Симулятор

TOPLEVEL_LANG = verilog

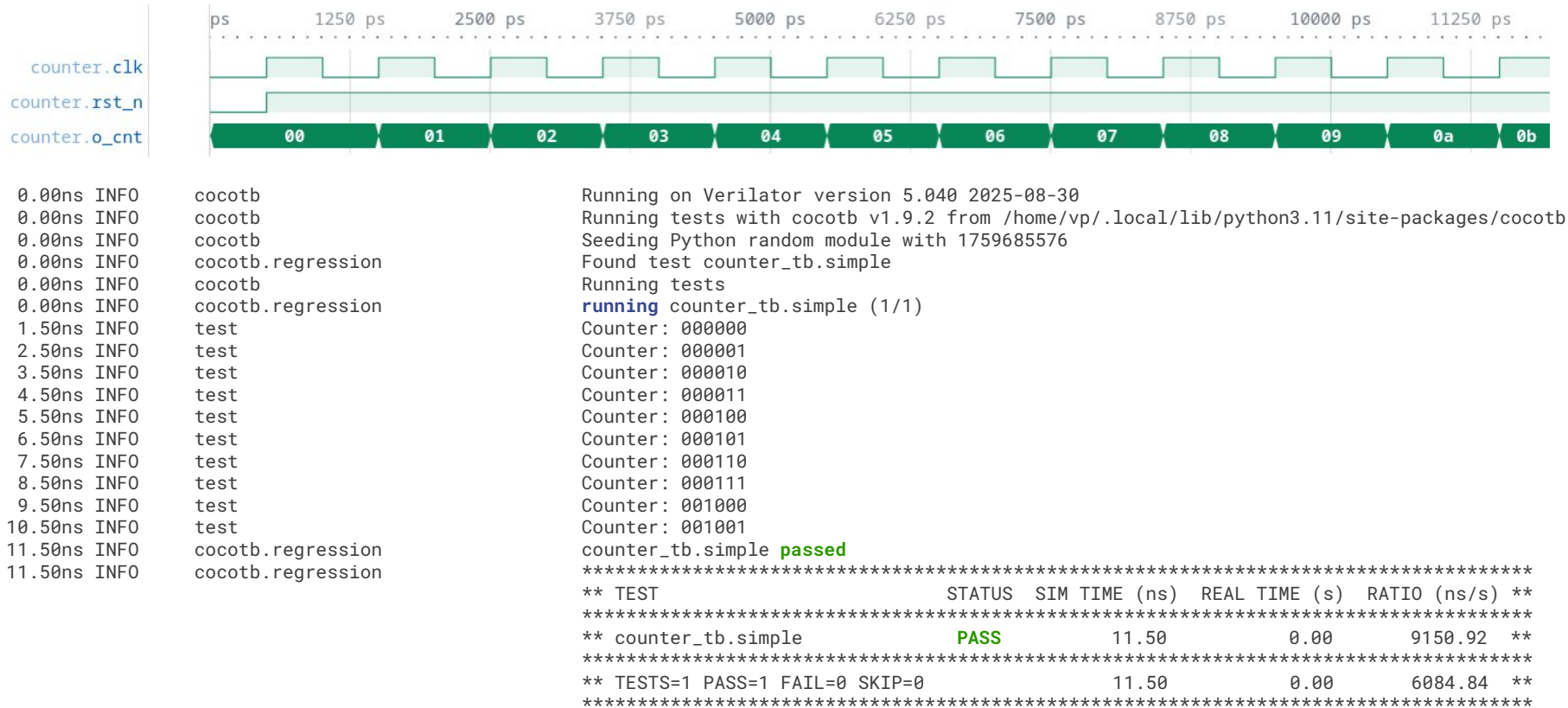
VERILOG_SOURCES += \
    src/counter.sv      # Список HDL-файлов

TOPLEVEL = counter      # Тестируемый RTL-модуль
MODULE    = counter_tb  # Тестбенч на Python

ifeq ($(SIM), verilator) # Сохранение всех сигналов в .fst файл
    EXTRA_ARGS += --trace --trace-fst --trace-structs
endif

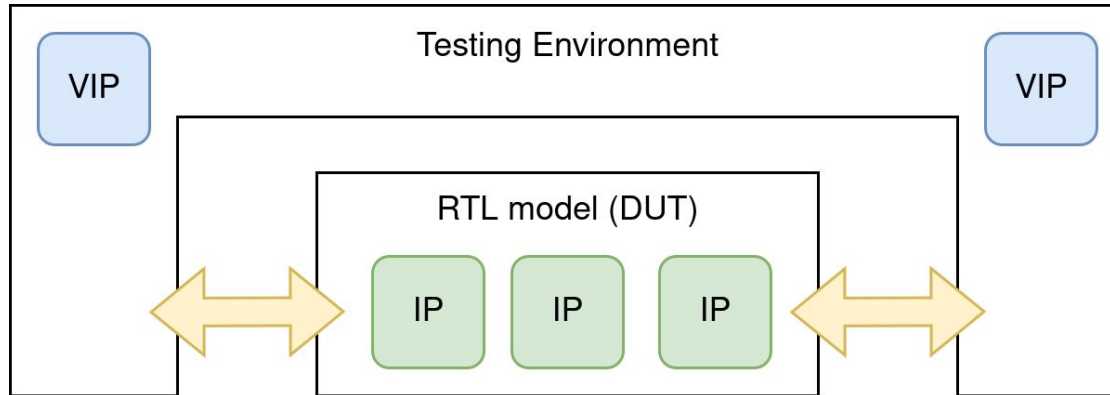
include $(shell cocotb-config --makefiles)/Makefile.sim # Makefile из cocotb
```

Запуск



Verification IP

- Разработка тестбенчей для каждого модуля с нуля может приводить к потере времени и ошибкам
- Переиспользовать можно не только **синтезируемое IP**, но и **верификационное IP (VIP)**



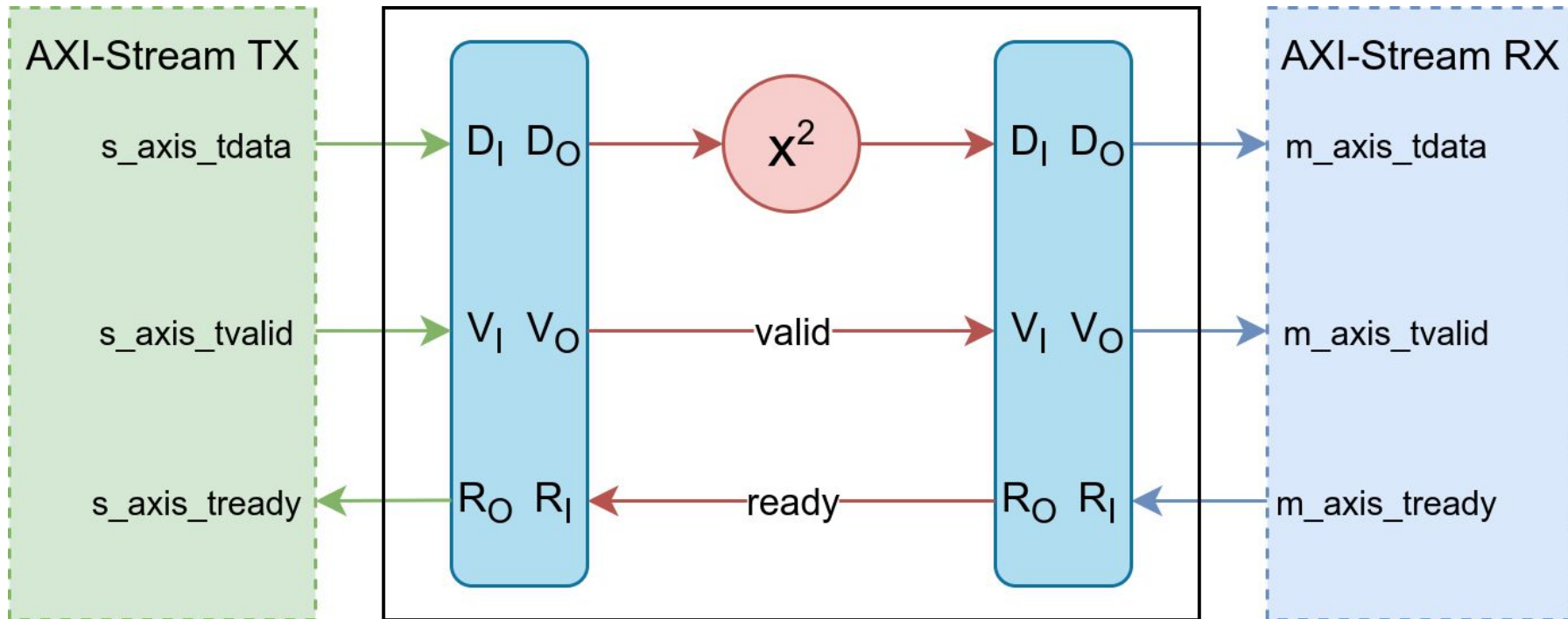
cocotbext-axi

- AXI-Stream
- AXI
- AXI-Lite
- APB

cocotbext-axi:AXI-Stream

- `AxiStreamBus` – объект AXI-Stream интерфейса
 - `from_prefix`
- `AxiStreamFrame` – объект AXI-Stream фрейма
 - `tdata, tkeep, tid, tdest, tuser`
- `AxiStreamSource` – модель AXI-Stream передатчика
 - `send, empty, full, wait, set_pause_generator`
- `AxiStreamSink` – модель AXI-Stream приемника
 - `recv, empty, full, wait, set_pause_generator`
- `AxiStreamMonitor` – модуль пассивного мониторинга на AXI-Stream шине

Пример



DUT: sqr.sv

```
module sqr #(
    parameter WIDTH = 16
) (
    input logic clk,
    input logic rst_n,

    input logic s_axis_tvalid,
    output logic s_axis_tready,
    input logic [WIDTH-1:0] s_axis_tdata,

    output logic m_axis_tvalid,
    input logic m_axis_tready,
    output logic [2*WIDTH-1:0] m_axis_tdata
);

logic x_vld;
logic x_rdy;
logic [WIDTH-1:0] x_data;
logic [2*WIDTH-1:0] x2_data;

assign x2_data = x_data * x_data;
```

```
peb #(.WIDTH(WIDTH)) peb0(
    .clk      (clk),
    .rst_n    (rst_n),
    .i_vld    (s_axis_tvalid),
    .o_rdy    (s_axis_tready),
    .i_data   (s_axis_tdata),
    .o_vld    (x_vld),
    .i_rdy    (x_rdy),
    .o_data   (x_data)
);

peb #(.WIDTH(2*WIDTH)) peb1(
    .clk      (clk),
    .rst_n    (rst_n),
    .i_vld    (x_vld),
    .o_rdy    (x_rdy),
    .i_data   (x2_data),
    .o_vld    (m_axis_tvalid),
    .i_rdy    (m_axis_tready),
    .o_data   (m_axis_tdata)
);

endmodule
```

TB: sqr_tb.py

```
from cocotbext.axi import AxiStreamBus, AxiStreamSource, AxiStreamSink
```

```
class TB:
```

```
    def __init__(self, dut):  
        self.dut = dut
```

```
        self.tx = AxiStreamSource(  
            AxiStreamBus.from_prefix(dut, "s_axis"),  
            dut.clk,  
            dut.rst_n,  
            reset_active_level=False  
        )
```

```
        self.rx = AxiStreamSink(  
            AxiStreamBus.from_prefix(dut, "m_axis"),  
            dut.clk,  
            dut.rst_n,  
            reset_active_level=False  
        )
```

```
        self.clock = Clock(dut.clk, 1, units="ns")
```

```
        cocotb.start_soon(self.clock.start())
```

TB: sqr_tb.py

```
from cocotbext.axi import AxiStreamBus, AxiStreamSource, AxiStreamSink
```

```
class TB:
```

```
    def __init__(self, dut):  
        self.dut = dut
```

```
        self.tx = AxiStreamSource(  
            AxiStreamBus.from_prefix(dut, "s_axis"),  
            dut.clk,  
            dut.rst_n,  
            reset_active_level=False  
        )
```

```
        self.rx = AxiStreamSink(  
            AxiStreamBus.from_prefix(dut, "m_axis"),  
            dut.clk,  
            dut.rst_n,  
            reset_active_level=False  
        )
```

```
        self.clock = Clock(dut.clk, 1, units="ns")
```

```
        cocotb.start_soon(self.clock.start())
```

TB: sqr_tb.py

```
from cocotbext.axi import AxiStreamBus, AxiStreamSource, AxiStreamSink

class TB:
    def __init__(self, dut):
        self.dut = dut

        self.tx = AxiStreamSource(
            AxiStreamBus.from_prefix(dut, "s_axis"), # Сигналы шины начинаются с s_axis_
            dut.clk,
            dut.rst_n,
            reset_active_level=False
        )

        self.rx = AxiStreamSink(
            AxiStreamBus.from_prefix(dut, "m_axis"), # Сигналы шины начинаются с m_axis_
            dut.clk,
            dut.rst_n,
            reset_active_level=False
        )

        self.clock = Clock(dut.clk, 1, units="ns")

        cocotb.start_soon(self.clock.start())
```

TB: sqr_tb.py: simple

```
SIZE = 2
NR_ITERS = 16
TIMEOUT = 500000

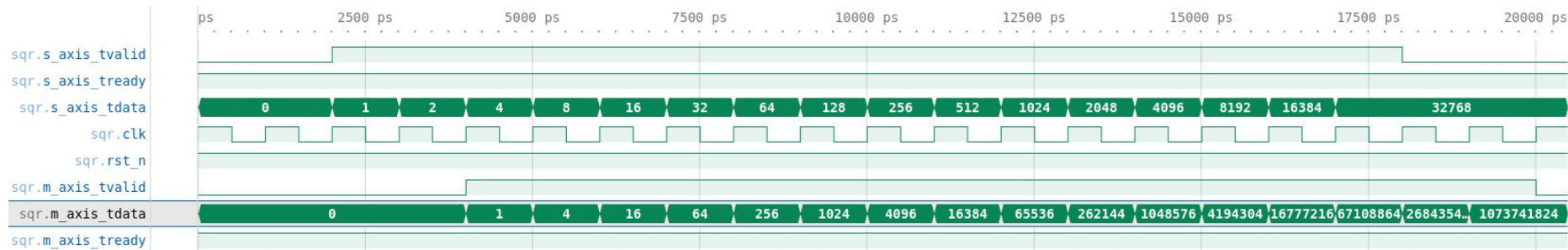
@cocotb.test(timeout_time=TIMEOUT, timeout_unit="ns")
async def simple(dut):
    tb = TB(dut)
    await tb.reset()

    for i in range(NR_ITERS):
        x = 1 << (i % 16)
        pkt = x.to_bytes(SIZE, 'little')
        await tb.tx.send(pkt)

    for i in range(NR_ITERS):
        x = 1 << (i % 16)
        pkt = (await tb.rx.recv()).tdata
        x2 = int.from_bytes(pkt, 'little')
        assert x2 == x**2

    assert tb.tx.empty()
    assert tb.rx.empty()
```


Пример: simple



```

3.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x01\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=3000, sim_time_end=None)
4.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x02\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=4000, sim_time_end=None)
5.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x04\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=5000, sim_time_end=None)
6.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x08\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=6000, sim_time_end=None)
6.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x01\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=6000, sim_time_end=6000)
7.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x10\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=7000, sim_time_end=None)
7.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x04\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=7000, sim_time_end=7000)
8.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=8000, sim_time_end=None)
8.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x10\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=8000, sim_time_end=8000)
9.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'@\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=9000, sim_time_end=None)
9.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'@\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=9000, sim_time_end=9000)
10.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x80\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=10000, sim_time_end=None)
10.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x01\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=10000, sim_time_end=10000)

```

```

16.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00 '), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=16000, sim_time_end=None)
16.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x10\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=16000, sim_time_end=16000)
17.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00@'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=17000, sim_time_end=None)
17.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00@x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=17000, sim_time_end=17000)
18.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00x80'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=18000, sim_time_end=None)
18.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00\x01'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=18000, sim_time_end=18000)
19.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00\x04'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=19000, sim_time_end=19000)
20.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00\x10'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=20000, sim_time_end=20000)
21.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=21000, sim_time_end=21000)

```

TB: sqr_tb.py: stress

```
from random import randint

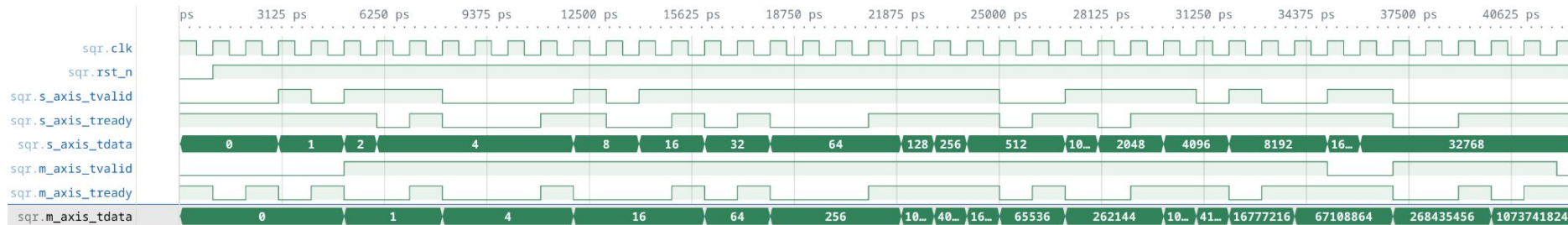
def gen_rand_bit():
    while True:
        yield randint(0, 1)

@cocotb.test(timeout_time=TIMEOUT, timeout_unit="ns")
async def stress(dut):
    tb = TB(dut)
    tb.tx.set_pause_generator(gen_rand_bit())
    tb.rx.set_pause_generator(gen_rand_bit())
    await tb.reset()

    for i in range(NR_ITERS):
        x = 1 << (i % 16)
        pkt = x.to_bytes(SIZE, 'little')
        await tb.tx.send(pkt)

    for i in range(NR_ITERS):
        x = 1 << (i % 16)
        pkt = (await tb.rx.recv()).tdata
        x2 = int.from_bytes(pkt, 'little')
        assert x2 == x**2
```

Пример: stress



```
3.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x01\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=3000, sim_time_end=None)
4.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x02\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=4000, sim_time_end=None)
6.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x04\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=6000, sim_time_end=None)
8.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x08\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=8000, sim_time_end=None)
8.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x01\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=8000, sim_time_end=8000)
12.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x04\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=12000, sim_time_end=12000)
13.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x10\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=13000, sim_time_end=None)
16.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=16000, sim_time_end=None)
16.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x10\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=16000, sim_time_end=16000)
18.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=18000, sim_time_end=None)
18.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'@\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=18000, sim_time_end=18000)
22.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x80\x00'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=22000, sim_time_end=None)
22.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x01\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=22000, sim_time_end=22000)
.....
31.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00 '), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=31000, sim_time_end=None)
31.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x10\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=31000, sim_time_end=31000)
32.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=32000, sim_time_end=32000)
33.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00@'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=33000, sim_time_end=None)
34.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00\x01'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=34000, sim_time_end=34000)
35.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00\x04'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=35000, sim_time_end=35000)
36.00ns INFO cocotb.sqr.s_axis TX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x80'), tkeep=None, tid=None, tdest=None, tuser=None, sim_time_start=36000, sim_time_end=None)
40.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00\x10'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=40000, sim_time_end=40000)
42.00ns INFO cocotb.sqr.m_axis RX frame: AxiStreamFrame(tdata=bytearray(b'\x00\x00\x00@'), tkeep=[], tid=[], tdest=[], tuser=[], sim_time_start=42000, sim_time_end=42000)
```

Задание

1. Разработать синхронное FIFO с интерфейсом AXI-Stream
2. Разработать тестбенч с проверкой Idle и Backpressure для AXI-Stream FIFO
3. Разработать тестбенч с проверкой Idle и Backpressure для систолического массива

Ссылки

- Документация cocotb: <https://docs.cocotb.org/en/stable/index.html>
- README для cocotbext-axi: <https://github.com/alexforencich/cocotbext-axi/blob/master/README.md>